

Cache Replication as a Liveness Signal: Unifying State Sharing and Failure Detection in LLM Service Clusters

A short technical note on the CHORUS mesh design

Amin Parva

InsightIts · 2026

Abstract

Clusters of stateless language-model services independently recompute answers to semantically identical requests, and independently maintain the failure detectors and consensus machinery that keep them available. These are normally two separate subsystems with two separate costs. We describe a design in which **a single peer-to-peer message stream serves both purposes at once**: the traffic that replicates cached answers between nodes is also the traffic that detects when a node has failed. A warm standby therefore arises as a *side effect* of cache sharing rather than as a dedicated mechanism, eliminating the separate heartbeat subsystem and the external consensus store that conventional architectures require. We situate this idea relative to prior work in semantic caching, gossip-based failure detection, and leaderless failover, and discuss its guarantees and limits.

1. Motivation

A language-model service deployed for availability runs as several replicas. Two independent inefficiencies follow from this replication:

Redundant computation. Replicas do not share results, so semantically equivalent requests routed to different replicas each incur the full cost of inference. The cluster pays k times for an answer it could compute once.

Redundant coordination. To remain available, the cluster also runs a failure-detection and failover mechanism — typically heartbeats plus a consensus protocol (Raft, or a managed equivalent), or an orchestrator's liveness probes. This is a second distributed subsystem, with its own operational surface.

The observation motivating this note is that **these two subsystems are carrying correlated information**. A replica that shares its computed results with peers is, by the very act of sharing, continuously signalling that it is alive. If peers consume that signal, the dedicated heartbeat becomes redundant.

2. The design

Each node maintains peer connections over which it emits small typed messages ("frames"). Among the frame types, two are relevant here: one carries a newly computed answer to be cached by all peers (state replication), and one carries a periodic node-health snapshot (liveness). Crucially, **both are emitted on the same channel by the same active node**, so the *absence* of either stream is evidence of failure.

The cluster runs a tiered topology:

- An **active** node serves requests and broadcasts each computed answer.
- A **warm standby** continuously receives those broadcasts. Its cache is therefore already populated with the active node's results. It also observes the liveness stream.
- A **reserve** node, typically in a separate failure domain, does the same.

When the active node's stream goes silent beyond a threshold, the warm standby promotes itself. Because its state was kept current by the replication traffic all along, promotion requires no state transfer — the standby is warm *precisely because* it was receiving cache updates. The detection mechanism and the state-synchronisation mechanism are the same mechanism.

3. Why this is different from prior art

The constituent ideas are individually well known; the contribution is their unification.

Semantic caching of model outputs is established practice, but existing systems treat the cache as a per-process or externally-stored artifact. They do not use cache-replication traffic for any purpose beyond caching.

Gossip and heartbeat failure detectors (e.g. SWIM-style protocols, phi-accrual detectors) are a mature field, but they transport *dedicated* liveness information; the payload exists only to prove aliveness.

Leaderless and quorum failover (Raft, Sentinel-style promotion) provides standby election, but the standby's state currency is a separate concern, handled by replication logs or snapshots distinct from the liveness channel.

In each case the literature keeps *state replication* and *failure detection* in separate planes. The design here **collapses them**: the replication payload is the liveness payload's carrier, so the warm standby is a free by-product of caching rather than a separately engineered capability. To the best of our knowledge this specific unification — for language-model service clusters, where the replicated state is exactly the expensive-to-recompute cached answer — is not described in prior systems. It is the subject of a provisional patent filing.

4. Guarantees and limits

We are precise about what the design does and does not provide.

It provides warm-standby failover with no dedicated heartbeat subsystem and no external consensus store, at the cost only of the cache-replication traffic that is already being sent.

It does not provide the strong-consistency guarantees of a quorum protocol. Promotion is leaderless and threshold-based; under a network partition the design admits a transient window in which two nodes consider themselves active. This is acceptable for a *cache* — a briefly duplicated active node wastes some recomputation but cannot corrupt a system of record — and is explicitly *not* appropriate as a consistency mechanism for durable state.

The failure-detection latency is dominated by a deliberately conservative silence threshold, chosen to avoid promoting on transient network jitter. This is a tunable trade-off between detection speed and false-positive rate, not a fundamental property.

5. Empirical observation

A reference deployment of this design — three nodes across two cloud virtual networks — exhibited the expected behavior: standby nodes served requests the active node had already answered with no recomputation; inter-node liveness and state frames crossed virtual-network boundaries with low, stable latency; and a silenced active node was succeeded by a warm standby in time dominated by the chosen silence threshold, with the promotion step itself completing in well under a tenth of a second. Detailed measurements appear in the companion benchmark report; they are not reproduced here, as this note concerns the design rather than its performance characterisation.

6. Conclusion

When the state a cluster must replicate is also the state whose freshness makes a standby promotable, the replication channel can do double duty as the failure detector. For language-model service clusters — where the replicated state is the costly cached answer — this unification removes an entire coordination subsystem. The design trades quorum-grade consistency for operational simplicity, a trade that is appropriate for caching workloads and inappropriate for systems of record. We believe the unification itself is novel and offer this note as a concise description of it.

Correspondence: insightits.info@gmail.com. A companion benchmark report and the open-source reference implementation (Apache 2.0) are available on request.